

# Push\_swap — The Complete Guide (aligned with your official subject PDF, v1.1)

**Correction from earlier:** I initially flagged this project as using fabricated requirements, based on the *generic* 42 push\_swap subject used at most campuses. Your actual subject PDF is different — it's a Warsaw-specific variant that genuinely requires a group of 2, four selectable strategies, a disorder metric, and a `--bench` mode. I was wrong to dismiss that. Everything below is rebuilt directly from your uploaded PDF, cross-checked line by line.

## Table of Contents

1. [Project shape at a glance](#)
  2. [Prerequisites — lock these down first](#)
  3. [The two-stack mental model](#)
  4. [Data structure design](#)
  5. [The disorder metric — full derivation](#)
  6. [The four required strategies, in depth](#)
  7. [The adaptive dispatcher](#)
  8. [CLI, flags, and program behavior](#)
  9. [Benchmark mode \(`--bench`\)](#)
  10. [Error handling — exact rules](#)
  11. [The step-by-step build plan](#)
  12. [The checker \(bonus\)](#)
  13. [README.md — exact required structure](#)
  14. [Group project mechanics & defense](#)
  15. [Testing methodology](#)
  16. [AI usage — what the subject actually says](#)
  17. [AI usage disclosure for this guide](#)
- 

## 1. Project shape at a glance

**Mandatory:** one binary, `push_swap`, that embeds all four strategies.

```
./push_swap [--simple|--medium|--complex|--adaptive] [--bench] <numbers...>
```

- Takes a list of integers as stack (`a`) (first argument = top of stack), no duplicates.
- Optionally takes one strategy flag. Default, if none given, is `--adaptive`.
- Optionally takes `--bench`, which prints metrics to **stderr** after sorting (operations still go to stdout).

- Prints the shortest operation sequence you can manage to stdout, one op per line, `(\n)`-separated, nothing else.
- On any invalid input: `(Error\n)` to `stderr`, no partial output.
- On zero arguments: print nothing, return the prompt — this is not an error case.

**This is a group project — exactly 2 learners**, both expected to understand and be able to defend every part of the code, not just the half you personally wrote.

**Bonus:** `(checker)` — reads operations from stdin, applies them to a stack built from the same argument list, prints `(OK)`/`(KO)`/`(Error\n)`. **Only graded if your mandatory part is fully perfect** — meaning every benchmark target below is met with zero exceptions. This is a hard gate, not a soft recommendation: if even one of your `(--bench)` numbers misses a target, the bonus isn't looked at at all.

**Performance targets** (measured with the provided checker during evaluation):

| Input size | Pass        | Good       | Excellent  |
|------------|-------------|------------|------------|
| 100 values | < 2000 ops  | < 1500 ops | < 700 ops  |
| 500 values | < 12000 ops | < 8000 ops | < 5500 ops |

Since bonus eligibility depends on hitting these cleanly across the board, **don't treat "excellent" as a stretch goal — treat it as your actual target from day one**, especially since your adaptive strategy has to cover all three complexity regimes and any one of them underperforming could be what's tested against you.

## 2. Prerequisites

Same foundational gate as any stack/pointer-heavy project, but this one leans harder on Big-O intuition than libft or GNL did, because you're being asked to *justify* four different complexity classes by name.

### 2.1 Stacks

LIFO — last in, first out. You only ever touch the top. Given your VFX layer-stack background, this maps almost directly: you can only act on the top of the undo history, never reach into the middle.

**Exercise:** by hand, given `(a = [3, 1, 4, 1, 5])` (3 on top — pretend duplicates are fine just for this drill), trace the state after: `push(9)`, `pop()`, `push(2)`, `pop()`, `pop()`. Do this before writing a line of code.

### 2.2 Circular doubly linked lists

You've already built singly linked list tooling in libft. The delta here:

- **Doubly linked** `((next) + (prev))` →  $O(1)$  traversal backward, which is what makes `(rra)` as cheap as `(ra)`.

- **Circular** (last→first, first→last) → rotation becomes "move which node the stack calls top," not "shift every element."

#### Four-question pass:

- *What:* a DLL whose ends wrap around to each other instead of terminating at `(NULL)`.
- *Why generally:*  $O(1)$  traversal both directions,  $O(1)$  rotation.
- *Why for this project:* `(ra)`/`(rra)` are your single most-called operation family across every strategy — if these cost  $O(n)$  instead of  $O(1)$ , your op *counts* stay the same but your program's *runtime* becomes painful at  $n=500$ , and worse, it signals to an evaluator that you didn't think about the underlying cost model.
- *How:* design the node struct and the four rotate functions first, test them in total isolation with a 4-5 element list, print the stack after every call, before any sorting logic touches them.

### 2.3 Big-O, made concrete against *this subject's numbers*

The subject explicitly tells you complexity is measured in **push\_swap operations generated**, not classical array-op complexity — that's an unusual and important framing, and it's the crux of why the four-strategy structure exists at all. Concretely:

- $O(n^2)$  for  $n=100$  informally implies "roughly  $n$  operations per element,  $n$  times" — a scan-and-move-minimum approach naturally lands around 5,000–10,000 ops. This clears none of your three tiers on its own at  $n=100$ , which is exactly why it's gated to only fire when disorder is *already* low (§7).
- $O(n \cdot \sqrt{n})$  for  $n=100$ :  $\sqrt{100}=10$ , so  $\approx 10$  operations per element  $\times 100 \approx 1,000$  ops in the right ballpark — this is your bridge strategy, meant to land in the "pass" to "good" range.
- $O(n \log n)$  for  $n=100$ :  $\log_2(100) \approx 6.6$ , so  $\approx 7$  operations per element  $\times 100 \approx 700$  — right at your "excellent" boundary, which is not a coincidence; it's why the complex strategy is reserved for the highest-disorder (hardest) case.

**Exercise:** before reading §6, write down your own back-of-envelope estimate for how many total operations a "find min, rotate to top, push to b, repeat" approach costs for  $n=500$ , using the same per-element  $\times n$  reasoning. Compare it against the 12,000/8,000/5,500 targets.

### 2.4 C mechanics you'll lean on

- `(malloc)`/`(free)` discipline on every parse path, success or error.
- `(strtol)` (not `(atoi)`) for overflow-safe integer parsing — you've done this carefully before in `ft_printf/libft`.
- Comfortable iteration over linked structures, since "find index of value" and "find cheapest rotation direction" get called constantly across every strategy.

### 2.5 Prerequisite gate

Don't move to §3 until, from memory:

1. You can draw a 4-node circular doubly linked list with every `(next)`/`(prev)` labeled.

2. You can explain why  $\boxed{ra}/\boxed{rra}$  are  $O(1)$  on this structure.
3. You can explain, in your own words, why the subject measures complexity in *push\_swap operations* rather than array comparisons — and why that distinction matters for how you'll argue your README's complexity claims.

### 3. The two-stack mental model

(see the first diagram above)

Two stacks,  $\boxed{a}$  (starts full, unsorted, no duplicates) and  $\boxed{b}$  (starts empty). Eleven legal moves:

| Op            | Effect                                                                                                                     |
|---------------|----------------------------------------------------------------------------------------------------------------------------|
| $\boxed{sa}$  | swap top 2 of $\boxed{a}$ (no-op if $<2$ elements)                                                                         |
| $\boxed{sb}$  | swap top 2 of $\boxed{b}$                                                                                                  |
| $\boxed{ss}$  | $\boxed{sa} + \boxed{sb}$ together — <b>counts as one operation</b> , always prefer this over two lines when you need both |
| $\boxed{pa}$  | pop top of $\boxed{b}$ , push onto $\boxed{a}$ (no-op if $\boxed{b}$ empty)                                                |
| $\boxed{pb}$  | pop top of $\boxed{a}$ , push onto $\boxed{b}$ (no-op if $\boxed{a}$ empty)                                                |
| $\boxed{ra}$  | rotate $\boxed{a}$ up: first element becomes last                                                                          |
| $\boxed{rb}$  | rotate $\boxed{b}$ up                                                                                                      |
| $\boxed{rr}$  | $\boxed{ra} + \boxed{rb}$ together, one operation                                                                          |
| $\boxed{rra}$ | reverse rotate $\boxed{a}$ : last element becomes first                                                                    |
| $\boxed{rrb}$ | reverse rotate $\boxed{b}$                                                                                                 |
| $\boxed{rrr}$ | $\boxed{rra} + \boxed{rrb}$ together, one operation                                                                        |

The subject's own worked example (§VI.4 of your PDF) sorts  $\boxed{[2, 1, 3, 6, 5, 8]}$  in 12 operations using  $\boxed{sa}$ , three  $\boxed{pb}$ , a combined  $\boxed{rr}$ , a combined  $\boxed{rrr}$ , another  $\boxed{sa}$ , and three  $\boxed{pa}$ . That example is worth re-tracing by hand on paper before you write code — it's the cleanest illustration of "use  $\boxed{b}$  as scratch space, always prefer the combined op when both stacks need the same move."

**Exercise:** re-derive that 12-operation solution yourself from scratch, physically crossing out and rewriting both stacks after each line, without looking at the PDF's answer first. Then check whether you can find a shorter sequence — the subject literally asks "can you do better?"

## 4. Data structure design

**Global variables are explicitly forbidden** by the subject — so your stack state has to be threaded through function parameters (or a single struct you pass by pointer), not sitting at file scope.

Recommended: circular doubly linked list, with a precomputed rank field.

```
c
typedef struct s_node
{
    int          value;
    int          index;    // rank if the whole set were sorted, 0..n-1
    struct s_node *next;
    struct s_node *prev;
} t_node;

typedef struct s_stack
{
    t_node *top;
    int     size;
} t_stack;
```

Four-question pass on **index**:

- *What*: a precomputed rank — 0 for the smallest value in the whole input,  $n-1$  for the largest.
- *Why generally*: turns "is this the minimum remaining" or "which chunk does this belong to" into an  $O(1)$  integer comparison instead of an  $O(n)$  value scan, every single time any strategy asks it — and every strategy in §6 asks it constantly.
- *Why for this project specifically*: it's also your clean way to sidestep negative numbers entirely for the radix strategy (§6.3) — you radix-sort on **index** (always  $0..n-1$ , unsigned-safe) rather than on the raw signed **value**.
- *How*: right after parsing and duplicate/overflow validation, copy all values into a temporary array, sort that copy, then walk your linked list once mapping each node's **value** to its position in the sorted copy.

Since the project is forbidden from global state, plan your function signatures early — e.g. every operation function takes **(t\_stack \*a, t\_stack \*b)**, and your benchmark counters (§9) get threaded through a struct rather than living as file-scope globals.

---

## 5. The disorder metric

Your PDF gives this exactly, and it's mandatory — you must compute it **before any operation is executed**:

```

function compute_disorder(stack a):
    mistakes = 0
    total_pairs = 0
    for i from 0 to size(a)-1:
        for j from i+1 to size(a)-1:
            total_pairs += 1
            if a[i] > a[j]:
                mistakes += 1
    return mistakes / total_pairs

```

This is literally **normalized inversion count**: for every pair of positions  $(i, j)$  with  $i$  before  $j$  in the stack, count it as a "mistake" if the earlier one is bigger than the later one. Divide by the total number of pairs,  $n(n-1)/2$ .

#### Four-question pass:

- *What*: a 0-1 score, 0 = perfectly sorted already, 1 = maximally reversed.
- *Why generally*: it's the standard measure of how far a sequence is from sorted order — related directly to how much work an adaptive sort needs to do (this is the same idea behind why insertion sort is fast on nearly-sorted data and slow on reversed data).
- *Why for this project*: it's the single number your adaptive dispatcher (§7) branches on — get this computation wrong and every downstream strategy selection is wrong too, even if each individual strategy is coded correctly.
- *How*: this is  $O(n^2)$  by construction (a full double loop over all pairs) — for  $n=500$  that's ~125,000 comparisons, trivially fast in C, so don't over-engineer this part; a direct, literal translation of the pseudocode is exactly right. Where people go wrong is computing this *after* some setup operations have already run — the subject is explicit that it must be measured before any moves.

**Worked micro-example**: for  $a = [3, 1, 2]$  (3 on top): pairs are (3,1)→mistake, (3,2)→mistake, (1,2)→not a mistake. mistakes=2, total\_pairs=3, disorder =  $2/3 \approx 0.667$ .

**Exercise**: compute disorder by hand for  $[1, 2, 3, 4, 5]$  (should be 0) and for  $[5, 4, 3, 2, 1]$  (should be 1). Then for the subject's own example input  $[2, 1, 3, 6, 5, 8]$  — this doubles as a check on your understanding of both this section and the worked example in §3.

## 6. The four required strategies, in depth

The subject is explicit: complexity here is measured in **push\_swap operations generated**, not classical algorithmic complexity translated 1:1. Keep that framing in your head for every subsection below — you're not implementing "insertion sort," you're implementing "a sequence-of-moves generator whose *count* happens to scale like insertion sort would."

### 6.1 Simple — $O(n^2)$

Pick one baseline: selection-style is the most commonly chosen because it's the easiest to reason about move-by-move.

**Approach:** repeatedly find the position of the current minimum remaining value in  $(a)$  (using your  $(index)$  field — the minimum remaining has the lowest untouched  $(index)$ ), rotate it to the top via whichever of  $(ra)/(rra)$  is cheaper (compare its distance from the top going up vs. going down), then  $(pb)$  it. After all elements have moved to  $(b)$  — where they end up in descending discovery order — rotate/ $(pa)$  them back into  $(a)$  in a way that reconstructs ascending order.

**Why  $O(n^2)$  in op-count terms:** each of the  $n$  passes costs roughly  $O(n)$  — an  $O(n)$  scan to find the minimum's position, plus  $O(n)$  worst-case rotation distance — so  $n$  passes  $\times O(n)$  per pass  $\approx O(n^2)$  total operations.

### Implementation tips:

- Handle the trivial cases first (see below) so this function is never called on  $n \leq 3$ .
- This strategy should be your very first fully-working end-to-end version (see §11 step 4) — it exists to prove correctness before you chase operation counts.

## 6.2 Medium — $O(n \cdot \sqrt{n})$ (chunk-based / "Turkish algorithm")

This is the standard approach most 42 students converge on for the middle tier, and it's explicitly listed in your PDF as an accepted example.

1. Use the  $(index)$  field from §4 to know each value's rank  $0..n-1$ .
2. Split the rank range into  $(k \approx \sqrt{n})$  chunks (for  $n=100$ ,  $\sim 10$  chunks of  $\sim 10$  each is a reasonable starting point — this is a tuning knob, not a fixed constant).
3. Process chunks from lowest rank to highest. Within a chunk: repeatedly find the cheapest-to-reach element in  $(a)$  whose rank falls in the current chunk (comparing  $(ra)$  distance vs  $(rra)$  distance, taking whichever is fewer moves),  $(pb)$  it.
4. Once a chunk is exhausted from  $(a)$ , its elements sit in  $(b)$  in the order you pushed them. Rotate/ $(pa)$  them back into  $(a)$  such that they land correctly ordered relative to chunks already placed.
5. Repeat for the next chunk.

**Why  $O(n \cdot \sqrt{n})$ :** each of the  $n$  elements gets touched roughly  $(\sqrt{n})$  times across the  $k \approx \sqrt{n}$  chunk passes (one scan-and-move per chunk it's "visible" during), rather than  $n$  times as in §6.1 — giving  $n \cdot \sqrt{n}$  total.

**This is where most debugging time goes.** Get one chunk working correctly on a small, fixed 10-20 element test set before trusting the loop across all chunks (see §11 step 7).

## 6.3 Complex — $O(n \log n)$ (radix sort with two stacks)

Your PDF lists radix (LSD or MSD) first among the accepted examples, and it's the cleanest to reason about precisely.

**Core idea:** sort by the binary representation of each value's  $(index)$  (not the raw signed value — this is exactly why §4's rank field exists), bit by bit, least-significant first.

```

for bit from 0 to (bits needed to represent n-1):
    for each element currently in a, top to bottom:
        if (index >> bit) & 1 == 0:
            ra                                // keep in a, rotate it to the bottom
        else:
            pb                                // push to b
    while b is not empty:
        pa                                // push everything back, restoring group order

```

After processing all necessary bits,  $a$  is sorted ascending. Since you're radix-sorting on  $\text{index} \in [0, n-1]$  rather than the raw  $\text{value}$ , you never touch the sign bit or worry about negative numbers at all — that complexity is absorbed entirely by the rank-mapping step in §4.

**Why  $O(n \log n)$ :** you do  $\lceil \log_2 n \rceil$  passes (not a fixed 32-bit pass — only as many bits as needed to represent  $n-1$ , which matters a lot for keeping this genuinely competitive rather than doing far more passes than necessary), and each pass touches every element once, giving  $n \cdot \log_2(n)$  total.

#### Implementation tips:

- Compute the number of bits needed once:  $\text{bits} = \text{ceil}(\log_2(n))$ , not a hardcoded 32 — using 32 when  $n=100$  only needs 7 bits would blow your operation count by roughly 4-5x for no reason.
- This is naturally the strategy most likely to hit your "excellent" tier (§1) given the §2.3 math, so it's worth the extra care.

#### 6.4 Trivial cases ( $n \leq 3$ ) — implement regardless of which strategy is active

- $n=2$ : one  $\text{sa}$  if out of order, else nothing.
- $n=3$ : exactly six possible orderings exist, and each has a known short fixed sequence (rotations/swaps) that never exceeds 3 operations. Hard-code this as a lookup table rather than running any general strategy on it — running a "real" algorithm on 3 elements wastes moves and is exactly the kind of thing a defense question would probe.

---

## 7. The adaptive dispatcher

(see the second diagram above)

This is strategy #4, and it's the one the subject leaves most open to your own design — but it comes with **hard thresholds you must respect**:



| Disorder           | Required internal complexity |
|--------------------|------------------------------|
| $d < 0.2$          | $O(n^2)$                     |
| $0.2 \leq d < 0.5$ | $O(n\sqrt{n})$               |
| $d \geq 0.5$       | $O(n \log n)$                |

### Four-question pass:

- *What:* a single function that computes disorder (§5) once, then calls into whichever of §6.1/6.2/6.3 matches the threshold table.
- *Why generally:* nearly-sorted input genuinely doesn't need an  $O(n \log n)$  sledgexhammer — a cheap  $O(n^2)$  pass will already be fast in absolute terms when disorder is low, since the constant factor per "mistake" is what dominates, not the asymptotic class.
- *Why these specific thresholds:* 0.2 and 0.5 aren't derived from anything you need to prove mathematically — the subject explicitly says you design the internal techniques, but you must **document your rationale for the thresholds in the README** (§13). A defensible answer: at  $d < 0.2$  there are relatively few inversions, so an  $O(n^2)$  selection-style approach's actual operation count stays low in practice even though its asymptotic class is worse; as disorder rises, the cost of scan-and-move-minimum grows faster than chunk- or radix-based approaches, so the crossover points are where empirical operation counts on your own test data actually favor switching.
- *How:* implement this as the very last strategy (see §11 step 9) — it's a thin dispatcher over code you've already built and tested independently, so implement it last precisely because it has the least new logic and the most dependency on everything else being correct first.

**Practically:** run your own benchmark sweep (vary disorder synthetically — you can construct inputs with a target inversion count) and log actual operation counts per strategy at several disorder values. Your README's threshold justification should reference this data, not just restate the subject's own boundary numbers back at it.

## 8. CLI, flags, and program behavior

```
./push_swap [--simple|--medium|--complex|--adaptive] [--bench] <stack-a-as-integers>
```

- Stack (a)'s first argument is the top of the stack.
- At most one strategy flag may be given; `--adaptive` is the default if none is given.
- `--bench` is independent of the strategy flag and can be combined with any of them.
- The strategy selection must work correctly for **all valid inputs**, regardless of size or disorder — i.e. `--simple` forced on a 500-element reversed list must still produce a

*correct* sort (it's allowed to be operation-expensive in that case, since you forced a mismatched strategy, but it must not be wrong or crash).

- Operations print to stdout, one per line, `(\n)`-separated, nothing else — no trailing prompt text, no extra whitespace.
  - Zero arguments: no output at all, clean return — not an error.
- 

## 9. Benchmark mode (`--bench`)

When present, after sorting completes, print to **stderr** (never stdout — stdout must remain pure operation output so it can still be piped straight into `(checker)`):

- Computed disorder as a percentage with two decimals.
- The strategy name actually used, plus its complexity class.
- Total operation count.
- Per-operation-type counts: `(sa)`, `(sb)`, `(ss)`, `(pa)`, `(pb)`, `(ra)`, `(rb)`, `(rr)`, `(rra)`, `(rrb)`, `(rrr)`.

Your PDF's own example output format (worth matching closely, since a `[bench]`-prefixed line style is shown explicitly):

```
[bench] disorder: 40.00%
[bench] strategy: Adaptive / O(nv^n)
[bench] total_ops: 13
[bench] sa: 0 sb: 0 ss: 0 pa: 5 pb: 5
[bench] ra: 2 rb: 1 rr: 0 rra: 0 rrb: 0 rrr: 0
```

**Implementation note:** since global variables are forbidden, thread a counts struct through your operation functions (e.g. every `(sa())`/`(pb())`/etc. call increments a field on a struct passed by pointer, the same struct also holding `(top_a)`/`(top_b)`) rather than reaching for a static/global tally.

---

## 10. Error handling — exact rules

Print `(Error\n)` to **stderr** and produce no stdout output for:

- Any argument that isn't a valid integer.
- Any integer outside `(int)` range (use `(strtol)` and check its bounds, not `(atoi)`).
- Any duplicate value.

No arguments at all is **not** an error — print nothing, return cleanly.

Free every node you've allocated so far on the error path before exiting — this is exactly what Valgrind and a careful evaluator both check first.

---

## 11. The step-by-step build plan

Work through in order — each step should produce something you can actually run.

1. **Node + raw operation functions.** `(sa)` through `(rrr)`, each only manipulating pointers and printing its own name. Test each in isolation on a tiny hardcoded 4-element list, printing the stack after every call.
  2. **Argument parsing + duplicate/overflow validation,** `(Error\n)` to stderr exact per §10, before any sorting logic exists.
  3. **Trivial-case lookup for  $n \leq 3$**  (§6.4). Verify against your own §3 hand-derivation.
  4. **Simple strategy (§6.1), unconditionally wired up** — get any input sorting correctly first, however operation-expensive. This is your correctness baseline for everything after.
  5. **Write the checker (bonus, §12) early**, even though it's only graded once mandatory is perfect — it becomes your primary correctness tool for every step from here on, much less painful to write against a known-working `(push_swap)` than blind at the end.
  6. **Compute the `(index)` field (§4).** Sanity-check by printing value+index pairs side by side on a small input by eye.
  7. **Implement disorder (§5)** as a standalone function, verified against the hand-worked examples in that section before it's wired to anything else.
  8. **Medium strategy (§6.2).** Fixed small chunk count on a fixed 15–20 element test input first; print stack state after every chunk completes; get one chunk fully correct before trusting the full loop.
  9. **Complex strategy (§6.3).** Verify bit count computation `((ceil(log2(n)))` in isolation before wiring the full radix loop.
  10. **Adaptive dispatcher (§7)** — thin glue over the three strategies you've already independently validated.
  11. **`(--bench)` mode (§9)** — thread the counts struct through every operation function.
  12. **Tune chunk size (medium) and confirm bit-count correctness (complex)** against the performance targets in §1, on both  $n=100$  and  $n=500$  random input.
  13. **Valgrind pass** — every malloc freed, success and error paths both.
  14. **README + defense prep** (§13, §14).
- 

## 12. The checker (bonus)

Reuses your stack + operation functions. Takes stack `(a)` as arguments (first = top). If given no arguments, stops and prints nothing. Then reads operations from stdin, one per line, `(\n)`-terminated, until EOF, applies each in order to the stack built from the arguments. After all instructions:

- If `(a)` is sorted ascending and `(b)` is empty  $\rightarrow$  `(0K\n)` to stdout.
- Otherwise  $\rightarrow$  `(K0\n)` to stdout.
- On any invalid input (non-integer arg, out-of-range, duplicate, unrecognized/malformed instruction)  $\rightarrow$  `(Error\n)` to **stderr**.

The subject explicitly notes you don't have to replicate the exact same parsing behavior as their reference binary — error handling is mandatory, but the specifics of how you validate arguments are yours to decide.

**Remember the gate from §1:** this bonus is only evaluated at all if your mandatory part passes every benchmark target with zero exceptions.

---

### 13. README.md — exact required structure

Per your PDF, the README must include, at minimum, in this order conceptually (not necessarily literal section order beyond the first line):

1. **First line, italicized, exact format:** `*This project has been created as part of the 42 curriculum by <login1>, <login2>.*`
2. **Description** — what the project is, its goal, brief overview.
3. **Instructions** — compilation, installation, execution.
4. **Resources** — classic references on the topic, *and* a description of how AI was used: for which tasks, and which parts of the project.
5. **Algorithm justification** — a detailed explanation and justification of the algorithms you selected for each of the four strategies, including your adaptive threshold rationale (§7) with the empirical reasoning behind 0.2 and 0.5, not just a restatement of the subject's own numbers.
6. Additional sections as useful (usage examples, feature list, technical choices) — not required but commonly expected in practice.

English is recommended, or your campus's main language.

---

### 14. Group project mechanics & defense

- **Exactly 2 learners**, both listed as contributors, both named in the README per §13.
  - Both must be present at defense and able to explain **any** part of the code — not just the half each of you wrote. Split the work, but cross-review each other's sections until you could each defend either half cold.
  - Expect a **live modification request** during evaluation — a small behavior change, a few lines to write or rewrite, something feasible in a few minutes. This is specifically designed to test whether you understand the code rather than memorized it, so the strongest preparation is genuinely understanding *why* each piece works, not just that it does.
  - Be ready to justify, without notes: why you chose the internal techniques you did for each of the four strategies, why your adaptive thresholds are where they are (with your own benchmark data, not just the subject's numbers), and how `(ss)`/`(rr)`/`(rrr)` combined ops are actually used in your code wherever applicable (an easy, embarrassing gap to have missed).
-

## 15. Testing methodology

```
bash

# random input generator
shuf -i 1-100 -n 100 | tr '\n' ' '

# operation count for one run
./push_swap $(shuf -i 1-100 -n 100 | tr '\n' ' ') | wc -l

# full correctness + count + bench loop
ARG=$(shuf -i 1-500 -n 500 | tr '\n' ' ')
./push_swap --bench --adaptive $ARG 2> bench.txt | ./checker $ARG
cat bench.txt
```

Script this into a loop across 50–100 random trials at  $n=100$  and  $n=500$ , logging op counts against the tiers in §1. Explicitly also test: already-sorted, fully-reversed, small  $n$  (0,1,2,3), and — since your adaptive dispatcher is threshold-based — inputs synthetically constructed to land just above and just below 0.2 and 0.5 disorder, to confirm the dispatcher actually branches where you think it does.

---

## 16. AI usage — what the subject actually says

Your PDF has a full chapter on this (Chapter III), and it's worth internalizing rather than skimming: use AI to reduce repetitive/tedious tasks, build prompting skill, understand how AI systems work — but **only use AI-generated content you fully understand and can take responsibility for**. Its own good/bad practice examples are pointed: asking "how do I test a sorting function" and refining ideas with a peer is fine; copy-pasting a generated function you can't explain during peer evaluation is an explicit failure case in the PDF itself. Given this is a defended, paired project with a live-modification evaluation step, that distinction isn't abstract — it's the actual failure mode the subject is warning you about by name.

---

## 17. AI usage disclosure for this guide

This guide was produced with AI assistance (Claude) at the user's request, as a study/reference document rather than project code, built directly from the user's uploaded official subject PDF (Push\_swap, v1.1) rather than generic external sources. An earlier draft of this guide incorrectly assumed the subject matched the standard cross-campus version and flagged the group/strategy/disorder requirements as fabricated; that was a mistake, corrected once the actual PDF was provided. As always: verify anything here against your own subject PDF and your teammate's understanding before relying on it in your defense.